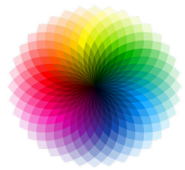


Dynamical Systems Meets Data:

DMD, Koopman, and Reduced Representations

Andrei Gavrilov and Nathan Mankovich



ISP • Image & Signal Processing
Universitat de València



Dynamical Systems

A **dynamical system** describes how a state $\mathbf{x}(t)$ evolves over time t according to a fixed rule.

$$\text{Continuous-time: } \frac{d\mathbf{x}}{dt} = g(\mathbf{x}(t)) \quad \text{Discrete-time: } \mathbf{x}(t + \tau) = f(\mathbf{x}(t))$$

- $\mathbf{x}(t) \in \mathbb{R}^n$: the state at time t
- f, g : a (possibly nonlinear) update rule
- Timeseries $\mathbf{x}(t)$
- Goal: Decompose the system into state space and temporal patterns, thus reducing temporal and state space dimensions.

The Simplest Dynamical System

First order, linear, ordinary differential equation.

$$\frac{d\mathbf{x}}{dt} = \lambda \mathbf{x}(t)$$

Closed form solution

$$\mathbf{x}(t) = e^{\lambda t} \mathbf{x}(0)$$

Discrete time formulation.

$$\mathbf{x}(t + \tau) = e^{\lambda \tau} \mathbf{x}(t)$$

The Simplest Dynamical System

$$\frac{d\mathbf{x}}{dt} = \lambda \mathbf{x}(t)$$

- $\lambda = 0 \implies$ no change
- $\lambda > 0 \implies$ exponential growth
- $\lambda < 0 \implies$ exponential decay

Linear Dynamical System

System of first order, linear, ordinary differential equation.

$$\frac{d\mathbf{x}}{dt} = \mathbf{L}\mathbf{x}(t)$$

Closed form solution

$$\mathbf{x}(t) = e^{\mathbf{L}t}\mathbf{x}(0)$$

Discrete time formulation.

$$\mathbf{x}(t + \tau) = e^{\mathbf{L}\tau}\mathbf{x}(t) = \mathbf{A}\mathbf{x}(t)$$

Solutions evolve through powers of $\mathbf{A}$: eigenvalues/eigenvectors govern behavior.

Dynamic Mode Decomposition

Assume that the data is sampled from the timeseries:

$$\mathbf{x}(t + \tau) = \mathbf{A} \mathbf{x}(t)$$

Decompose the system into spatial and temporal patterns.

Analyzing $\mathbf{A}$ results in the DMD (\citep{schmid2010dynamic}):

$$\mathbf{x}(t) = \sum_{j=1}^k \phi_j e^{\omega_j t} b_j$$

- $\phi_j \in \mathbb{C}^n$ **feature patterns** (dynamic modes)
- $\omega_j \in \mathbb{C}$ **temporal characteristics** (continuous time eigenvalues)
- $b_j \in \mathbb{R}$ scalar loadings (a.k.a. amplitudes)

Dynamic Mode Decomposition (math steps)

Starting from

$$\mathbf{x}(t + \tau) = \mathbf{A}\mathbf{x}(t)$$

iterate in discrete time ($m = t/\tau$):

$$\mathbf{x}(t) = \mathbf{A}^m \mathbf{x}(0)$$

If $\mathbf{A}\mathbf{W} = \mathbf{W}\mathbf{\Lambda}$ and $\mathbf{x}(0) = \mathbf{W}\mathbf{b}$, then

$$\mathbf{x}(t) = \mathbf{W}\mathbf{\Lambda}^m \mathbf{b} = \sum_{j=1}^k \phi_j \lambda_j^{t/\tau} b_j = \sum_{j=1}^k \phi_j e^{\omega_j t} b_j$$

Eigenvalue Interpretation

$$\lambda_j = e^{\omega_j \tau}$$

- Discrete time: λ_j
- Continuous time: ω_j

Useful conversion:

$$\omega_j = \frac{1}{\tau} \log(\lambda_j)$$

Eigenvalue Interpretation (polar form)

Write in polar form:

$$\lambda_j = r e^{i\theta}$$

- Trend: $r = |\lambda_j|$
- Oscillation angle per sample step: $\theta = \arg(\lambda_j)$

and with $\lambda_j = r e^{i\theta}$:

$$\omega_j = \frac{\log r + i\theta}{\tau}$$

Reading Frequency And Decay From Eigenvalues

For a chosen sampling lag τ , write the continuous-time eigenvalue as

$$\omega_j = \sigma_j + i\nu_j$$

with

$$\sigma_j = \frac{\log |\lambda_j|}{\tau}, \quad \nu_j = \frac{\arg(\lambda_j)}{\tau}.$$

- Oscillation frequency: $f_j = \frac{|\nu_j|}{2\pi} = \frac{|\arg(\lambda_j)|}{2\pi\tau}$
- Decay time (when $\sigma_j < 0$): $\tau_{\text{decay}}(\tau) = -\frac{1}{\sigma_j} = -\frac{\tau}{\log |\lambda_j|}$
- Growth time (when $\sigma_j > 0$): $\tau_{\text{grow}}(\tau) = \frac{1}{\sigma_j} = \frac{\tau}{\log |\lambda_j|}$

Exact DMD (Dawson et al.)

1. Stack the data

$$\mathbf{X} = [\mathbf{x}(1) \mid \cdots \mid \mathbf{x}(T - \tau)] \in \mathbb{R}^{n \times p}, \quad \mathbf{X}' = [\mathbf{x}(1 + \tau) \mid \cdots \mid \mathbf{x}(T)] \in \mathbb{R}^{n \times p}$$

2. Want to solve

$$\min_{\mathbf{A}} \|\mathbf{X}' - \mathbf{A}\mathbf{X}\|_F$$

3. Rank- r truncated SVD $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ leads to

$$\begin{aligned} \mathbf{A}_\star &= \mathbf{X}'\mathbf{X}^\dagger, \quad \mathbf{X}^\dagger \approx \mathbf{V}\mathbf{\Sigma}^{-1}\mathbf{U}^\top \\ \tilde{\mathbf{A}} &= \mathbf{U}^\top \mathbf{X}'\mathbf{V}\mathbf{\Sigma}^{-1} \in \mathbb{R}^{k \times k}, \quad \tilde{\mathbf{A}}\mathbf{W} = \mathbf{W}\mathbf{\Lambda} \end{aligned}$$

Exact DMD (continued)

4. Dynamic modes

$$\phi_j = \frac{1}{\lambda_j} \mathbf{X}' \mathbf{V} \mathbf{\Sigma}^{-1} \mathbf{w}_j$$

5. Eigenvalues (discrete time)

$$\lambda_j = e^{\omega_j \tau}$$

6. Loadings found by solving

$$\Phi \mathbf{b} = \mathbf{x}(1)$$

Compact reconstruction form:

$$\mathbf{x}(t) \approx \Phi \operatorname{diag}(e^{\omega t}) \mathbf{b}$$

My Interpretation (easier?)

The system evolves in the low-dimensional reduced space mapped to by $\mathbf{U}^\top \in \mathbb{R}^{k \times n}$:

$$\mathbf{U}^\top \mathbf{x}(t + \tau) = \mathbf{z}(t + \tau) = \tilde{\mathbf{A}} \mathbf{z}(t) = \tilde{\mathbf{A}} \mathbf{U}^\top \mathbf{x}(t)$$

Eigen-decompose $\mathbf{A}$ into eigenvectors $\mathbf{w}_j$ and eigenvalues λ_j .

Projected dynamic modes:

$$\phi_j = \mathbf{U} \mathbf{w}_j$$

DMD analyzes the stability of this system and generates spatial patterns in $\mathbb{R}^n$ (ambient space).

Optimized DMD (Askham et al.)

Stack all the data together:

$$\mathbf{X} = [\mathbf{x}(t_1) \mid \mathbf{x}(t_2) \cdots \mid \mathbf{x}(t_p)] \in \mathbb{R}^{n \times p}$$

Optimize directly for DMD matrix decomposition:

$$\min_{\phi_j, \omega_j, b_j} \left\| \mathbf{X} - \sum_{j=1}^r \phi_j e^{\omega_j t} b_j \right\|_F$$

Translate to:

$$\min_{\mathbf{A}} \|\mathbf{X}^\top - \mathbf{\Phi}(\boldsymbol{\alpha})\mathbf{B}\|_F$$

Optimized DMD (continued)

Equivalent elementwise model:

$$\mathbf{x}_{i,:}^\top \approx \sum_{j=1}^r e^{\alpha_j t_i} \mathbf{B}_{j,:}$$

- $\Phi(\alpha)_{i,j} = \exp(\alpha_j t_i)$
- $b_j = \|\mathbf{B}^\top(:, j)\|_2$
- $\phi_j = \frac{\mathbf{B}^\top(:, j)}{b_j}$

Solve via variable projection method.

DMD with Control (DMDc)

If external inputs influence the dynamics, model the system as

$$\mathbf{x}(t + \tau) = \mathbf{A}\mathbf{x}(t) + \mathbf{B}\mathbf{u}(t).$$

Given state snapshots $\mathbf{X}$, $\mathbf{X}'$ and input snapshots $\mathbf{U}$, estimate both operators by

$$\min_{\mathbf{A}, \mathbf{B}} \|\mathbf{X}' - \mathbf{A}\mathbf{X} - \mathbf{B}\mathbf{U}\|_F.$$

- DMDc extends DMD from autonomous systems to controlled systems.
- Same idea: fit a linear operator from data, then analyze or use it for prediction and control.

Further Reading

There are MANY variants of DMD

- Multiverse of DMD
- DMD with control (DMDc)
- Physics-informed DMD
- Generalizing DMD: Modern Koopman theory

Transition To Koopman Lab

- DMD gives the core idea: estimate a linear operator from data and analyze its spectrum.
- For nonlinear systems, we keep that same idea but move from state space to observable space.
- In the lab, the main object will be a finite-dimensional Koopman/EDMD fit:

$$\mathbf{K} = \arg \min_{\mathbf{K}} \|\Psi_X \mathbf{K} - \Psi_Y\|_F^2.$$

- So the next section is not a separate topic: it is the nonlinear extension of the DMD story.

Tutorial

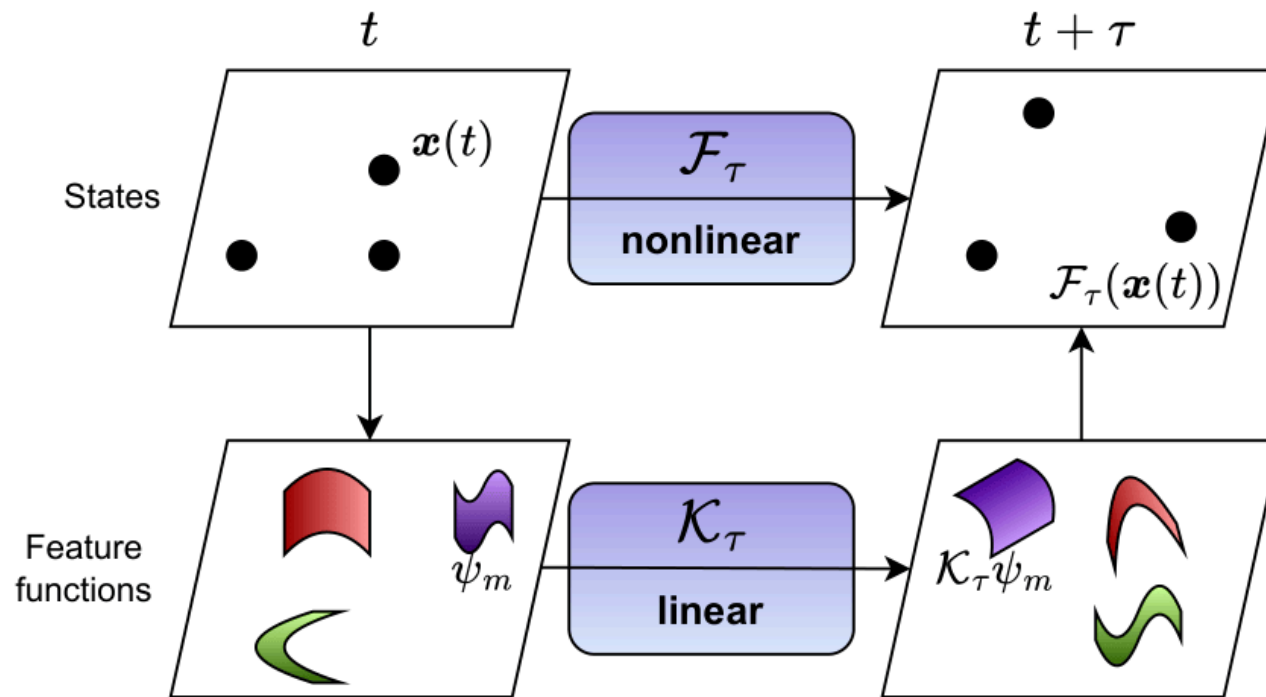
<https://github.com/PyDMD/PyDMD/blob/master/tutorials/tutorial1/tutorial-1-dmd.ipynb>

Koopman Mode Decomposition

Koopman Mode Decomposition

Problem: there's no linear map that goes from $\mathbf{x}(t)$ to $\mathbf{x}(t + \tau)$...

Solution:



Linear Koopman = DMD

If we choose the observable map to be the identity,

$$\psi(\mathbf{x}) = \mathbf{x},$$

then the finite-dimensional Koopman update becomes

$$\psi_{k+1} = \psi_k \mathbf{K} \implies \mathbf{x}_{k+1} = \mathbf{x}_k \mathbf{K}.$$

So for linear observables, the Koopman matrix is exactly the same linear evolution operator used in DMD:

$$\mathbf{K} = \mathbf{A}.$$

- DMD is the special case of Koopman analysis with no lifting.
- Koopman methods generalize DMD by replacing $\mathbf{x}$ with richer observables $\psi(\mathbf{x})$.

Koopman Mode Decomposition

Choose an observable dictionary

$$\psi(\mathbf{x}) = [\psi_1(\mathbf{x}), \dots, \psi_m(\mathbf{x})]^\top.$$

Koopman operator (denoted $\mathcal{K}_\tau$) ([Koopman 1931](#)):

$$\mathcal{K}_\tau[\psi_j](\mathbf{x}(t)) = \psi_j(\mathcal{F}_\tau \mathbf{x}(t)) = \psi_j(\mathbf{x}(t + \tau))$$

- The state dynamics $\mathcal{F}_\tau$ may be nonlinear.
- The Koopman operator is linear in the observables, even when $\mathcal{F}_\tau$ is not.

Koopman Mode Decomposition (finite-dimensional view)

Williams et al. 2015, Klus et al. 2017

In the notebook, we approximate the Koopman operator on a finite dictionary of observables using EDMD.

Given snapshots $\mathbf{x}_k$ and next-step snapshots $\mathbf{y}_k$, define row-stacked lifted data matrices

$$\mathbf{\Psi}_X = \begin{bmatrix} \psi(\mathbf{x}_1)^\top \\ \vdots \\ \psi(\mathbf{x}_N)^\top \end{bmatrix}, \quad \mathbf{\Psi}_Y = \begin{bmatrix} \psi(\mathbf{y}_1)^\top \\ \vdots \\ \psi(\mathbf{y}_N)^\top \end{bmatrix}.$$

Koopman Mode Decomposition (finite-dimensional fit)

We fit a finite-dimensional Koopman matrix $\mathbf{K}$ by

$$\mathbf{K} = \arg \min_{\mathbf{K}} \|\Psi_X \mathbf{K} - \Psi_Y\|_F^2.$$

The notebook also learns a decoder back to state space,

$$\hat{\mathbf{x}} = \psi(\mathbf{x}) \mathbf{B}.$$

- $\mathbf{K}$ advances observables forward in time.
- $\mathbf{B}$ maps lifted coordinates back to physical variables.
- This is the exact pipeline used in the lab before spectral analysis.

Koopman Spectral Analysis

In the notebook, the fitted Koopman matrix is analyzed through

$$\mathbf{A} := \mathbf{K}.$$

We compute its right eigenvectors and eigenvalues:

$$\mathbf{A}\mathbf{V} = \mathbf{V}\mathbf{\Lambda}, \quad \mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_r).$$

- λ_j are discrete-time Koopman eigenvalues
- $\omega_j = \tau^{-1} \log(\lambda_j)$ are continuous-time eigenvalues
- This is exactly the `eigvals, eigvecs_right = np.linalg.eig(A)` step in the notebook.

Koopman Eigenfunctions

For $\psi_{k+1} = \psi_k \mathbf{A}$, the Koopman eigenfunctions are

$$\phi_j(\mathbf{x}) = \psi(\mathbf{x}) \mathbf{v}_j.$$

On the training snapshots, stacking the eigenvectors into $\mathbf{V}$ gives

$$\Phi(X) = \Psi_X \mathbf{V}.$$

- The notebook computes this as `psi_train @ eigvecs_right`.
- These coordinates show how each spectral component evolves along the trajectory.

Koopman Dynamic Modes

The decoder maps observables back to the state space:

$$\hat{\mathbf{x}} = \psi(\mathbf{x}) \mathbf{B}.$$

Changing to the eigenvector basis gives

$$\hat{\mathbf{x}} = \Phi(\mathbf{x}) \mathbf{M}, \quad \mathbf{M} = \mathbf{V}^{-1} \mathbf{B}.$$

The columns of $\mathbf{M}^\top$ are the Koopman dynamic modes in the original state coordinates.

- This is the notebook step `modal_decoder = np.linalg.solve(eigvecs_right, koop_model.B_)`.
- Then `dynamic_modes = modal_decoder.T` stores the state-space modes columnwise.

Koopman Mode Decomposition

Putting the fitted operator, eigenfunctions, and decoder together gives

$$\hat{\mathbf{x}}_{k+s} = \psi(\mathbf{x}_k) \mathbf{A}^s \mathbf{B} = \sum_{j=1}^r \lambda_j^s \phi_j(\mathbf{x}_k) \mathbf{b}_j.$$

- $\phi_j(\mathbf{x})$ are Koopman eigenfunctions
- $\mathbf{b}_j$ are dynamic modes in the original state space
- λ_j controls growth, decay, and oscillation

Tutorial

- Extended (Kernel) DMD
- Koopman Mode Decomposition